

◆ GUNS 2 AMMO ◆

SYSTEM WORKFLOW

Booking, Payments, Check-in & Staff Operations

RELEASE V1.12.2

Prepared for client handover
Guns 2 Ammo — Arizona Shooting Range

Guns 2 Ammo — Booking & Staff System

Complete Workflow & Feature Reference (v1.12.2)

Part 1 — System status (pre-delivery audit)

SUBSYSTEM	STATUS	WHERE IT LIVES
Customer booking flow (online)	Working	<code>class-bookings-controller.php</code>
Stripe payments + webhook	Working, signature-verified, idempotent	<code>payments/class-stripe.php</code> , <code>rest/class-webhooks-controller.php:20</code>
PayPal / Fortis / Auth.Net / Pay-in-Store	Available, off-by-default	<code>payments/</code>
Email automation (8 event types)	Working, admin-editable templates	<code>modules/email-automation/class-email-engine.php</code>
24h + 2h reminder emails	Working, runs every 5 min	<code>modules/email-automation/class-email-cron.php</code>
Member self check-in via QR	Working (new in v1.12)	<code>frontend/class-shortcode-member-checkin.php</code>
Staff dashboard + lane map	Working	<code>frontend/class-shortcode-staff-console.php</code>
Waiver lookup (people + archive)	Working (fixed in v1.12)	<code>rest/class-staff-controller.php:198</code>
Memberistic integration (waiver + tier)	Reads both tables	<code>modules/memberistic/</code>
PMPro membership pricing	Working, rule-based discounts	<code>modules/pmpo-memberships/</code>
Hold expiry cleanup cron	Every 5 min	<code>class-activator.php:474</code>
Daily reports cron	Daily	<code>class-activator.php:476</code>
Customer self-reschedule / cancel (token-gated)	Working	<code>rest/class-calendar-controller.php</code>
Refund handling (Stripe)	Working via <code>charge.refunded</code> webhook	<code>payments/class-stripe.php:319</code>
Payment-amount cross-check	Refuses status flip on mismatch	<code>payments/class-stripe.php</code>
Roles + capabilities (4 staff roles)	Created on activation	<code>class-activator.php:114</code>
WP-Cron OPcache reset on activate	Working	<code>class-activator.php:45</code>
DB schema self-heal	Working (v1.12)	<code>class-installer.php</code>

Auto-renewal note: The booking plugin does **not** run renewal billing itself. Membership renewals are owned by Memberistic and/or PMPro (whichever is in use). The booking plugin only **reads** membership status to apply pricing and gate features. If you need renewal-reminder emails to members specifically about their *membership* (not bookings), that lives in Memberistic's own settings — confirm with the client that those are enabled there.

Part 2 — The system in one paragraph (video opener)

Guns 2 Ammo's booking engine is a complete reservation, payment, check-in, and member-management platform built into WordPress. Customers book lanes online and pay through Stripe. Members get tier-based pricing automatically. Staff run the entire front desk from a single dashboard that shows live lane status, pending check-ins, today's roster, and the full waiver database. Members can scan a QR on the dashboard to check themselves in from their phone, with staff confirming the lane assignment in one click. The system sends automated emails at every step — reservation receipt, payment receipt, 24-hour reminder, 2-hour reminder, check-in confirmation, cancellation, no-show — all template-editable from the WordPress admin.

Part 3 — Daily staff workflow (chronological)

3.1 Opening shift (staff arrives, ~9:50 AM)

1. Staff opens browser → goes to `guns2ammo.com/staff/`
2. Logs in with their WordPress account (role: `g2ab_manager` or `g2ab_staff`)
3. Dashboard loads showing: - **KPI strip** — Lanes In Use (e.g. `0 / 6`), Today's Revenue (\\$), Active Members count, Live Lanes - **Live Lane Map** — every lane as a color-coded tile (green = open, yellow = reserved, red = in-use) - **Live Feed** — real-time stream of recent events (payments, check-ins, no-shows)
4. Staff clicks **Check-In Station** in the sidebar
5. The page renders a large QR code. The QR is signed and valid for 12 hours (one full shift), then auto-rotates.
6. Staff opens the **Reservations** pane briefly to see today's bookings list.

3.2 Walk-in customer arrives without a reservation

Option A — staff initiates the walk-in:

1. Staff clicks the orange **Walk-in Check-In** button (top-right of dashboard)
2. Modal opens. Staff enters: name, email, phone, lane, party size, minutes (defaults 60)
3. Click **Check in** → `POST /staff/walk-in` creates a `checked_in` booking
4. KPIs and lane map refresh; lane tile turns red

Option B — customer scans the dashboard QR:

1. Customer scans the QR on their phone
2. Lands on `/check-in/?s=<token>` → sees a clean mobile form
3. If logged in to WordPress → identity is automatic ("Notify front desk" button)
4. If not → enters email + last name
5. Submits → their phone shows a spinner: *"Front desk has been notified"*
6. Dashboard pops a **confirm modal** within 2-3 seconds showing: - Waiver status banner (green = current, red = expired, gray = missing) - Name, email (masked), membership tier, waiver signed date, expiry - Lane picker (open lanes only) - Minutes (default 60), party size (default 1) - "Email a check-in receipt to the member" checkbox (default ON)

7. Staff picks lane → clicks **Confirm check-in**
8. Booking created with `status=checked_in` . Member's phone flips to "You're checked in" with lane + time + duration. Email receipt arrives in their inbox.
9. If their waiver is expired → staff can click **Decline** → member's phone shows "Please see the front desk" → staff handles in person.

3.3 Looking up a known member

1. Staff clicks **Waivers** in sidebar
2. Types name or email (e.g. `Nick.s@guns2ammo.com`)
3. Search hits BOTH `wp_memberistic_people` AND `wp_memberistic_waivers_archive` — results merged, deduped by email
4. Each result card shows: masked email, waiver status badge (current/expired/missing/archived), signed date, expiry date, membership tier
5. If the waiver is current, a **Check In →** button appears
6. Click it → same confirm modal as above, pre-filled → pick lane → confirm → checked in + email sent

3.4 Mid-shift — managing the live roster

1. Click **Reservations** in sidebar
2. Today's roster shows: time, name, lane, party size, status pill, and three action buttons: - ✓ Check in - × No-show - \ \$ Mark paid
3. Clicking any action calls `POST /staff/booking-action` → updates the booking → log row → toast confirmation → roster refresh

3.5 End-of-day

Staff signs out using **Sign Out** in the sidebar. The dashboard's auto-refresh stops. Behind the scenes, the daily reports cron (`g2ab_daily_reports`) compiles the day's metrics for the admin dashboard.

Part 4 — Customer journey (online booking)

1. Customer visits a booking page (e.g. `/book-a-lane/` rendering `[g2a_lane_booking]`)
2. Picks a date → sees available time slots (`GET /availability` — public endpoint)
3. Picks a slot → enters name, email, phone, party size
4. Picks a payment method (Stripe by default, optional PayPal / Fortis / Auth.Net / Pay-in-Store)
5. Submits → `POST /bookings` creates a booking with `status=reserved` (held for 15 minutes)
6. Redirected to Stripe Checkout
7. On successful payment, Stripe webhook hits `/wp-json/g2a-booking/v1/webhooks/stripe`
8. Plugin verifies the `Stripe-Signature` HMAC, marks booking as `paid` , inserts a row in `wp_g2ab_payments` (idempotent on `(gateway, transaction_id)`)
9. Fires `g2ab_payment_succeeded` and `g2ab_booking_status_changed` actions
10. Email engine listens and sends `booking_paid` email to customer (and admin if enabled)
11. If the customer abandons checkout → 15 minutes later the hold-expiry cron releases the slot

Part 5 — Email automation (every email the system can send)

EVENT	WHEN IT FIRES	WHO RECEIVES	EDITABLE?
<code>booking_created</code>	New reservation submitted	Customer + Admin	Yes — Admin → Settings → Email
<code>booking_confirmed</code>	Staff manually confirms (or auto on payment)	Customer	Yes
<code>booking_paid</code>	Stripe webhook returns success	Customer + Admin	Yes
<code>booking_cancelled</code>	Customer or staff cancels	Customer	Yes
<code>booking_no_show</code>	Staff marks no-show	Customer	Yes (off by default)
<code>booking_completed</code>	Booking end time passes	Customer	Yes
<code>booking_reminder_24h</code>	24 hours before start (cron)	Customer	Yes
<code>booking_reminder_2h</code>	2 hours before start (cron)	Customer	Yes
<code>member_checkin</code> (NEW v1.12)	Staff confirms a self check-in	Member	Yes (inline HTML in <code>finalize_checkin</code>)

How emails actually leave the server:

- Sent via `wp_mail()` — automatically uses any SMTP plugin the client has installed (FluentSMTP, WP Mail SMTP, etc.)
- From-name and From-address are admin-configurable
- HTML body wrapped in a brand-styled template with the business name and brand color
- **Kill switch:** an admin can globally disable all emails via the `g2ab_email_kill_switch` option
- **Duplicate protection:** the engine refuses to send the same event for the same booking twice (tracked in `wp_g2ab_logs`)
- Every send writes a log row → searchable in the admin's Logs view

Merge tags available in templates: `{customer_name}`, `{resource_name}`, `{start_at}`, `{end_at}`, `{amount}`, `{currency}`, `{booking_uuid}`, `{reschedule_url}`, `{cancel_url}`, `{business_name}`, `{brand_color}`, `{phone}`, `{address}`

Part 6 — Payment notifications (the chain)

When Stripe processes a payment, the system does **six** things in sequence:

1. Stripe POSTs the webhook to `/wp-json/g2a-booking/v1/webhooks/stripe` (no auth — public on purpose; the signature is the auth)
2. Plugin reads the `Stripe-Signature` header and verifies HMAC-SHA256 against the stored `whsec_` secret
3. Plugin reads `client_reference_id` (booking UUID) and looks up the booking
4. **Amount cross-check:** refuses to mark paid if `amount_total` $\neq$ `booking.total_amount` (logs a `payment_amount_mismatch` warning instead)
5. Marks booking `status=paid`, upserts a row in `wp_g2ab_payments` keyed by `(gateway, transaction_id)` so duplicate webhook deliveries don't double-credit
6. Fires `g2ab_payment_succeeded` → email engine sends `booking_paid` email to customer + admin

Refunds: Stripe sends `charge.refunded` → plugin's `mark_booking_refunded` finds the payment row, updates status to `refunded`, logs the event. Email template `booking_refunded` is **not** currently in the default template set — flag

for a future addition if the client wants automated refund emails.

Part 7 — Membership integration (how plans + bookings connect)

Two membership systems are supported in parallel:

Memberistic

- Plugin reads `wp_memberistic_people` and `wp_memberistic_waivers_archive`
- Surfaces waiver status (current/expired/missing) on every check-in
- Surfaces membership tier on the staff dashboard for context
- **Membership lifecycle (renewals, cancellations, expirations) is owned by Memberistic itself** — the booking plugin only reads the data

Paid Memberships Pro (PMPPro)

- If PMPPro is active, the plugin reads the customer's PMPPro level
- Applies discount rules per booking type (e.g. "Gold members get 20% off Lane bookings")
- "Members-only mode" available: a booking type can be restricted to specific PMPPro levels
- Rules are configured in **Settings → Memberships**

What the staff sees

- Tier label on every member card (e.g. "GOLD MEMBER")
- Member-pricing applied automatically when the member books online
- Walk-in pricing can be overridden by staff at check-in

Membership renewal emails

- If the client uses PMPPro, renewal reminders come from PMPPro's own settings
- If the client uses Memberistic, renewal reminders come from Memberistic's own settings
- The booking plugin does **not** override these — it doesn't need to

Part 8 — Roles & permissions

ROLE	WHAT THEY CAN DO
G2A Booking Manager (<code>g2ab_manager</code>)	Everything — bookings, resources, forms, payments, settings
G2A Booking Staff (<code>g2ab_staff</code>)	Manage bookings + view reports (no settings)
G2A Instructor (<code>g2ab_instructor</code>)	Manage bookings only
G2A Booking User (<code>g2ab_booking_user</code>)	Customer-facing role — view own bookings
Walk-in Customer (<code>g2a_walkin</code>)	Auto-created for walk-ins, no login

Who can see `[g2ab_staff_console]` : any logged-in user with the `manage_g2ab_bookings` capability. That means Manager, Staff, Instructor, or any Administrator (admins get all caps on activation).

Who can see `[g2ab_member_checkin]` : anyone with a valid station token in the URL. The station token is HMAC-signed and verified server-side; without it, the page shows "QR expired".

Part 9 — Scheduled background jobs

CRON	INTERVAL	PURPOSE
<code>g2ab_cleanup_expired_reservations</code>	Every 5 min	Releases held lanes from abandoned online checkouts (15-min hold)
<code>g2ab_email_reminder_tick</code>	Every 5 min	Scans for bookings 24h and 2h out, sends reminder emails
<code>g2ab_send_booking_reminders</code>	Hourly	Legacy hook (kept for backward compatibility)
<code>g2ab_daily_reports</code>	Daily	Compiles daily booking + revenue summary

All crons use WordPress's built-in `wp-cron`, so they fire on each page request. For production reliability, the client should set up a real OS-level cron hitting `wp-cron.php` every 5 minutes — standard WordPress best practice. (Mention this in the handover.)

Part 10 — Security posture (what to tell the client)

- **PII protection**: staff console never returns DOB, signed-PDF URL, raw IP, or waiver text — only masked email, name, status, and dates
 - **Stripe webhook**: HMAC-SHA256 signature verification with timing-safe comparison
 - **Member check-in API**: nonce-protected + rate-limited (30 requests/min/IP) + station token HMAC-validated on every submit
 - **Station tokens**: signed with 12-hour expiry, regenerated automatically
 - **Booking modification by customers**: token-gated URLs (`/bookings/{uuid}/customer-cancel`) — only the person who has the link can modify
 - **Payment idempotency**: UNIQUE KEY on `(gateway, transaction_id)` prevents double-credit from webhook retries
 - **Amount tampering**: payment refused if Stripe-reported amount doesn't match the booking total
 - **OPcache reset on activate**: prevents shared-host caching of old plugin bytecode after upgrades
-

Part 11 — Shortcode reference (every customer-facing surface)

SHORTCODE	RENDERS
[g2a_lane_booking]	Lane booking form
[g2a_classes_booking]	Classes booking form
[g2a_ladies_tuesday_booking]	Ladies' Tuesday booking form
[g2a_resource_booking]	Generic resource booking form
[g2a_booking_form]	Universal booking form
[g2a_events_list]	List of upcoming events
[g2a_events_calendar]	Calendar view of events
[g2a_events_carousel]	Carousel of events
[g2a_upcoming_events]	Compact upcoming-events widget
[g2a_event_countdown]	Countdown timer to next event
[g2a_event_banner]	Promotional event banner
[g2ab_reschedule]	Customer self-reschedule page (token-gated)
[g2ab_cancel_booking]	Customer self-cancel page (token-gated)
[g2ab_member_checkin]	Member self check-in (mobile) — NEW v1.12
[g2ab_staff_console]	Staff dashboard — NEW v1.11
[g2ab_frontdesk]	Legacy front-desk terminal

Part 12 — One-time setup checklist for the client

Confirm each of these is done before going live:

- [] Stripe live keys entered in **Settings → Payments → Stripe**
- [] Stripe webhook endpoint added in Stripe dashboard → `https://guns2ammo.com/wp-json/g2a-booking/v1/webhooks/stripe`
- [] Stripe webhook secret (`whsec_...`) pasted into **Settings → Payments → Stripe → Webhook secret**
- [] SMTP plugin installed and verified (FluentSMTP recommended)
- [] Email From-name + From-address set in **Settings → Email**
- [] Brand color set in **Settings → General** (drives `{brand_color}` in email templates)
- [] All 9 email templates reviewed and copy-edited
- [] Staff dashboard page created: a page containing `[g2ab_staff_console]` (e.g. `/staff/`)
- [] Check-in page auto-created by the plugin at `/check-in/` — verify it exists
- [] Staff user accounts created with `G2A Booking Manager` or `G2A Booking Staff` role
- [] OS-level cron hitting `wp-cron.php` every 5 minutes (replaces WP's default page-load-triggered cron)
- [] Test the full flow: book online → pay with Stripe test card → receive email → check in on dashboard → mark complete

Part 13 — Suggested video chapters

1. **00:00 — The big picture** (Part 2 paragraph)
 2. **00:30 — The staff dashboard tour** (Part 3, screen-record the four sidebar panes)
 3. **02:30 — Member self check-in (the QR flow)** (Part 3.2 Option B, record on a phone)
 4. **04:00 — Looking up a waiver and checking someone in** (Part 3.3)
 5. **05:30 — A customer books online** (Part 4, screen-record front-end booking)
 6. **07:00 — What emails go out and when** (Part 5 table)
 7. **08:00 — How payments work behind the scenes** (Part 6)
 8. **09:00 — Membership-aware pricing** (Part 7)
 9. **10:00 — Roles, security, and what runs automatically** (Parts 8–10)
 10. **11:30 — Wrap-up + handover checklist** (Part 12)
-

Part 14 — Known limitations to be transparent about

1. **QR library loads from CDN** (jsdelivr) at runtime. If the staff dashboard machine is offline or behind strict CSP, the QR won't render — falls back to displaying the URL as text. If the client wants fully air-gapped operation, this is a 30-min follow-up to vendor the library into the plugin.
 2. **No automated refund email** — refunds are processed end-to-end correctly, but the customer isn't auto-emailed. Easy to add if requested.
 3. **WP-Cron reliability** — depends on site traffic unless an OS cron is configured. Standard WordPress thing, but worth flagging.
 4. **Membership renewals** — owned by Memberistic/PMPPro, not by this plugin. Renewal emails come from those systems.
-

Document prepared for the Guns 2 Ammo handover. All claims verified against the v1.12.2 codebase.